lab08

AUTHOR
Otavio Ribeiro Mello

```
reticulate::py_require("polars")
```

```
import polars as pl
```

```
basics = (
    pl.scan_csv(
        "title.basics0.tsv.gz",
        separator="\t",
        null_values="\\N",
        quote_char=None,
        infer_schema_length=0,
    )
    .select(["tconst", "titleType", "primaryTitle", "startYear", "genres"])
    .with_columns(pl.col("startYear").cast(pl.Int32))
)

ratings = (
    pl.scan_csv(
        "title.ratings.tsv.gz",
        separator="\t",
        null_values="\\N",
        quote_char=None,
        infer_schema_length=0,
    )
    .with_columns(
        pl.col("averageRating").cast(pl.Float64),
        pl.col("numVotes").cast(pl.Int64),
    )
)

principals = (
    pl.scan_csv(
        "title.principals0.tsv.gz",
        separator="\t",
        null_values="\\N",
        quote_char=None,
        infer_schema_length=0,
    )
    .select(["tconst", "nconst", "category"])
)


filmes = basics.filter(pl.col("titleType") == "movie")
```

1. Quais são os 5 filmes com as maiores notas (`averageRating`)? Apresente uma solução capaz de desempatar os filmes baseando-se no número de votos recebidos.

```
q1 = (
    filmes
    .join(ratings, on="tconst", how="inner")
    .drop_nulls(["averageRating", "numVotes"])
    .sort(
        by=["averageRating", "numVotes", "tconst"],
        descending=[True, True, False],
    )
    .head(5)
    .select(["tconst", "primaryTitle", "startYear", "averageRating", "numVotes"])
    .collect()
)

print(q1)
```

```
shape: (5, 5)
┌────────────┬──────────────────────────┬───────────┬───────────────┬──────────┐
│ tconst     ┆ primaryTitle             ┆ startYear ┆ averageRating ┆ numVotes │
│ ---        ┆ ---                      ┆ ---       ┆ ---           ┆ ---      │
│ str        ┆ str                      ┆ i32       ┆ f64           ┆ i64      │
╞════════════╪══════════════════════════╪═══════════╪═══════════════╪══════════╡
│ tt33075815 ┆ Kaveri                   ┆ 2024      ┆ 10.0          ┆ 1023     │
│ tt27815015 ┆ Kurukku                  ┆ 2024      ┆ 10.0          ┆ 451      │
│ tt28450376 ┆ Jedal Dar Omghe 30 Metri ┆ 2009      ┆ 10.0          ┆ 142      │
│ tt27859713 ┆ Sargashte                ┆ 2016      ┆ 10.0          ┆ 134      │
│ tt33507675 ┆ Gorgeous Rascal          ┆ 2024      ┆ 10.0          ┆ 115      │
└────────────┴──────────────────────────┴───────────┴───────────────┴──────────┘
```

2. Qual é o gênero mais frequente entre os filmes com nota maior que 8?

```
q2 = (
    filmes
    .join(ratings.filter(pl.col("averageRating") > 8), on="tconst", how="inner")
    .select(["tconst", pl.col("genres").str.split(",")])
    .explode("genres")
    .drop_nulls("genres")
    .group_by("genres")
    .agg(pl.len().alias("n_filmes"))
    .sort(by=["n_filmes", "genres"], descending=[True, False])
    .head(5)
    .collect()
)

print(q2)
```

```
shape: (5, 2)
┌─────────────┬──────────┐
│ genres      ┆ n_filmes │
│ ---         ┆ ---      │
│ str         ┆ u32      │
╞═════════════╪══════════╡
│ Documentary ┆ 10564    │
│ Drama       ┆ 8114     │
```

```
| Comedy      | 3186     |
| Biography   | 1395     |
| Romance     | 1363     |
└─────────────┴──────────┘
```

3. Quais são os 3 atores/atrizes que mais participaram de filmes com nota maior que 7.5?

```
filmes_nota_alta = (
    filmes
    .join(ratings.filter(pl.col("averageRating") > 7.5), on="tconst", how="inner")
    .select("tconst")
)

q3 = (
    principals
    .filter(pl.col("category").is_in(["actor", "actress"]))
    .join(filmes_nota_alta, on="tconst", how="inner")
    .group_by("nconst")
    .agg(pl.col("tconst").n_unique().alias("n_filmes"))
    .sort(by=["n_filmes", "nconst"], descending=[True, False])
    .head(3)
    .collect()
)

print(q3)
```

```
shape: (3, 2)
┌────────────┬──────────┐
| nconst     | n_filmes |
| ---        | ---      |
| str        | u32      |
└────────────┴──────────┘
┌────────────┬──────────┐
| nm0004660  | 199      |
| nm0595934  | 155      |
| nm3183374  | 124      |
└────────────┴──────────┘
```

```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-10-14 21:08:29 -03"
```